$ W9 · 5GC DEPLOYMENT · FREE5GC

/> COMPOSE & HELM

從 Docker、Docker Compose 到 Kubernetes 與 Helm —— 把整套 5GC 搬進容器

/> 課程大綱

01

DOCKER

Dockerfile、build、push、up / down

02

AUTO RELEASE

CI 每日 latest 與 tag 版 image

03

COMPOSE

用 ULCL compose 拆解多容器

04

KUBERNETES

Node、Pod、Service、etcd、滾動更新

05

HELM & HA

Chart、values、Release、高可用

06

FREE5GC-HELM

Ansible 一鍵安裝 + k9s

Part 1 單機容器（01-03） · Part 2 叢集（04-05） · Part 3 實戰（06）

$ PART 1 · CONTAINER

/> DOCKER

把程式和它需要的一切打包成 image，在哪台機器都能跑起來

/> 為什麼需要容器？

沒有容器

Go / Node 版本跟同學不一樣

少裝了 iproute2、libsctp…

設定檔路徑、權限各自不同

15 個 NF 要一個一個編譯、啟動

「可是在我電腦上能跑啊！」

有容器

image 內含完整執行環境

一行 docker run 就能啟動

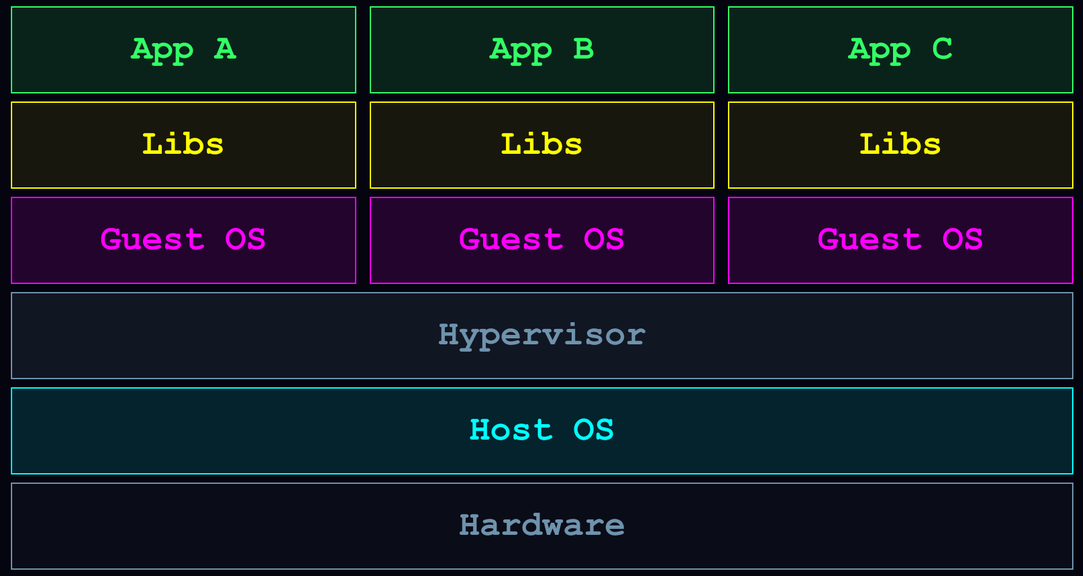
環境可重現、可版本化

彼此隔離，不污染 Host

「在哪台機器都一樣」

/> VM VS CONTAINER

Virtual Machine



Container



Container 沒有 Guest OS、直接共用 Host kernel → 更輕、啟動更快。也因此 UPF 需要的 `gtp5g` kernel module 必須裝在 Host 上。

/> 四個核心名詞

01 · RECIPE

Dockerfile

一份文字檔，描述 image 要怎麼一層一層組出來

02 · TEMPLATE

Image

唯讀、分層的模板；同一個 image 可以跑出很多 container

03 · INSTANCE

Container

image 跑起來的實例，有自己的檔案系統、網路與行程

04 · STORE

Registry

存放與分享 image 的倉庫，例如 Docker Hub

類比：Dockerfile 是食譜、Image 是冷凍料理包、Container 是加熱後端上桌的那一盤。

/> DOCKER 工作流程



接下來依序走一遍：① Dockerfile → ② Build → ③ Push → ④ Up → ⑤ Down

/> ① DOCKERFILE — FREE-RAN-UE

```
# ---------- Stage 1: 編譯 Go binary ----------
FROM golang:1.26.2 AS builder
WORKDIR /free-ran-ue
COPY free-ran-ue/ .
RUN go mod download
RUN make bin

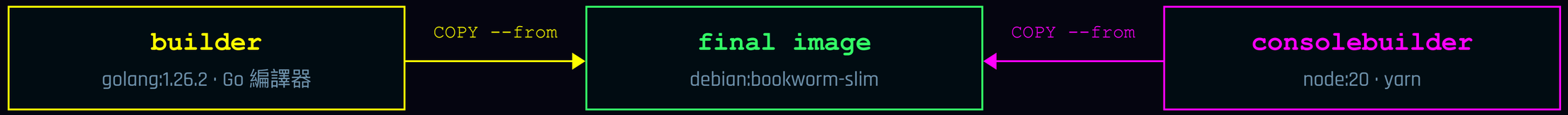
# ---------- Stage 2: 編譯 Web console ----------
FROM node:20 AS consolebuilder
WORKDIR /free-ran-ue
COPY free-ran-ue/ .
RUN cd console/frontend && yarn install
RUN make console

# ---------- Stage 3: 最終 image ----------
FROM debian:bookworm-slim
WORKDIR /free-ran-ue
RUN apt-get update && apt-get install -y iproute2 net-tools iputils-ping iperf3 curl
COPY --from=builder        /free-ran-ue/build/free-ran-ue /free-ran-ue/free-ran-ue
COPY --from=consolebuilder /free-ran-ue/build/console     /free-ran-ue/console
```

/> DOCKERFILE 常用指令

指令	作用
`FROM`	指定 base image，每個 stage 的起點
`WORKDIR`	設定之後指令的工作目錄
`COPY`	把檔案從 build context 複製進 image
`RUN`	build 時執行指令，每個 RUN 產生一層 layer
`CMD / ENTRYPOINT`	container 啟動時預設執行的指令
`EXPOSE / ENV`	宣告 port、設定環境變數
`AS / --from`	multi-stage：替 stage 命名，從別的 stage 拿產物

/> MULTI-STAGE BUILD



編譯工具只存在 builder stage，不會進入最終 image

最終 image 只留 binary、console 和少量除錯工具

image 更小 → pull 更快、攻擊面更小

/> ② BUILD DOCKER IMAGE

docker.io / **alonza0314** / **free-ran-ue** : **latest**

registry（可省略） namespace / 帳號 repository tag

```
$ cd fru-compose
$ git clone https://github.com/free-ran-ue/free-ran-ue
$ docker build -f Dockerfile -t alonza0314/free-ran-ue:latest .
$ docker images
REPOSITORY                TAG       IMAGE ID   CREATED   SIZE
alonza0314/free-ran-ue    latest    <id>       ...       ...
```

最後的 `.` 是 build context：COPY 只能拿到這個目錄底下的檔案。fru-compose 的 `make` 做的就是這行。

③ PUSH TO DOCKER HUB

```
$ docker login                                          # 登入 Docker Hub
$ docker tag  alonza0314/free-ran-ue:latest \
              alonza0314/free-ran-ue:v1.0.0              # 同一個 image 再貼一個 tag
$ docker push alonza0314/free-ran-ue:v1.0.0
$ docker push alonza0314/free-ran-ue:latest

# 在任何一台機器上
$ docker pull alonza0314/free-ran-ue:latest
```

tag 只是指向 image 的標籤，同一個 image 可以有很多 tag

`latest` 會一直被覆蓋；要固定行為就釘版本號

/> ④ UP CONTAINER

```
$ docker run -d --name ue \
    --cap-add NET_ADMIN --device /dev/net/tun \
    -v $(pwd)/config/uecfg.yaml:/free-ran-ue/uecfg.yaml \
    alonza0314/free-ran-ue:latest tail -f /dev/null
$ docker ps                    # 看正在跑的 container
$ docker logs -f ue            # 追蹤輸出
$ docker exec -it ue bash      # 進到 container 裡
```

-d / --name	背景執行、取名字	**-v**	掛載 Host 檔案 / volume
-p 5000:5000	Host port → container	**--cap-add**	給網路管理權限（建 TUN）

⑤ DOWN CONTAINER

```
$ docker stop ue            # 停止：先送 SIGTERM，逾時再 SIGKILL
$ docker start ue           # 停止的 container 可以再啟動
$ docker rm ue              # 刪除 container
$ docker rm -f ue           # 強制停止 + 刪除
$ docker rmi alonza0314/free-ran-ue:latest    # 刪除 image
$ docker system prune       # 清掉沒在用的 container / network / cache
```



container 被 rm 之後，裡面寫的資料就消失了 → 要保存的資料請放 volume。

$ PART 1 · CI / CD

/> AUTO RELEASE

free5GC 與 free-ran-ue 每天自動 release latest image，打 tag 時 release 特定版本

/> 為什麼讓 CI 來 BUILD？

使用者不用自己編譯，直接 `docker pull` 就能用

每個 image 都對應到明確的 source code 版本

在乾淨的環境 build，不會混入某人電腦上的東西

一次產出多架構：amd64 與 arm64（Mac / 樹莓派）

Docker Hub 帳密放在 GitHub Secrets，不落到個人手上

/> 兩種 TAG 策略

:latest

daily · schedule cron

- 每天固定時間自動觸發
- 抓 main / master 最新的程式碼
- 覆蓋同一個 latest tag
- 適合：開發、嘗鮮最新功能

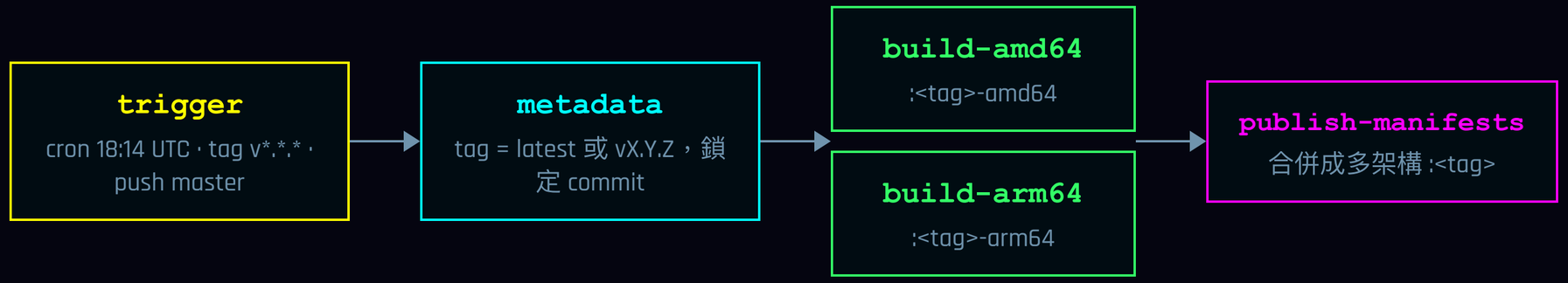
:v4.x.x

release · git tag

- 專案打 tag（如 v*.*.*）時觸發
- checkout 該 tag 的程式碼
- 產生固定不變的版本 tag
- 適合：教學、實驗、正式環境

compose / helm 裡寫 `:latest` 很方便，但結果會隨時間改變；要重現實驗請釘版本。

/> FREE5GC：RELEASE-IMAGES.YML




```
images=(nrf amf bsf scp smf udr pcf udm nssf ausf n3iwf upf chf tngf nef webui ueransim n3iwue)
docker buildx imagetools create --tag free5gc/amf:latest \
    free5gc/amf:latest-amd64 free5gc/amf:latest-arm64
```

使用者只要 pull `free5gc/amf:latest`，Docker 會依機器架構自動挑對的版本。

FREE-RAN-UE：DOCKER-RELEASE.YAML

```
on:
  schedule:
    - cron: '14 19 * * *'    # 每天 UTC 19:14
  workflow_dispatch:          # 也可手動觸發
env:
  IMAGE_NAME: alonza0314/free-ran-ue
  UPSTREAM_REPO: free-ran-ue/free-ran-ue
steps:
  - 取得 upstream 最新 release tag
  - Docker Hub 已有此 tag？→ 跳過
  - docker login (DOCKERHUB_TOKEN)
  - build.sh   <version>
  - release.sh <version>
```

build.sh

clone free-ran-ue → checkout 該 tag → docker build

release.sh

push :<version>，再 tag 成 :latest 一起 push

docker-build.yaml

每個 PR 都 build 一次並檢查 binary 在不在

冪等

已經 release 過的版本不會重複 build

$ PART 1 · MULTI-CONTAINER ON ONE HOST

/> COMPOSE

一個 YAML 描述整套 5GC + gNB + UE，一行指令全部起來

/> 一個一個 DOCKER RUN？

手動

```
$ docker network create sbi ...
$ docker network create cn-ran ...
$ docker network create ran-ue ...
$ docker run -d --name db  ... mongo
$ docker run -d --name nrf ... nrf
$ docker run -d --name amf ... amf
$ docker run -d --name smf ... smf
#   ... 還有 10 個 ...
#   順序錯了 → NF 註冊不到 NRF
```

Docker Compose

```
$ docker compose \
    -f docker-compose-ulcl.yaml \
    up -d
```

網路、volume 一起建立

依 depends_on 決定啟動順序

設定寫在檔案裡，可以 commit

/> DOCKER-COMPOSE-ULCL.YAML 拓樸

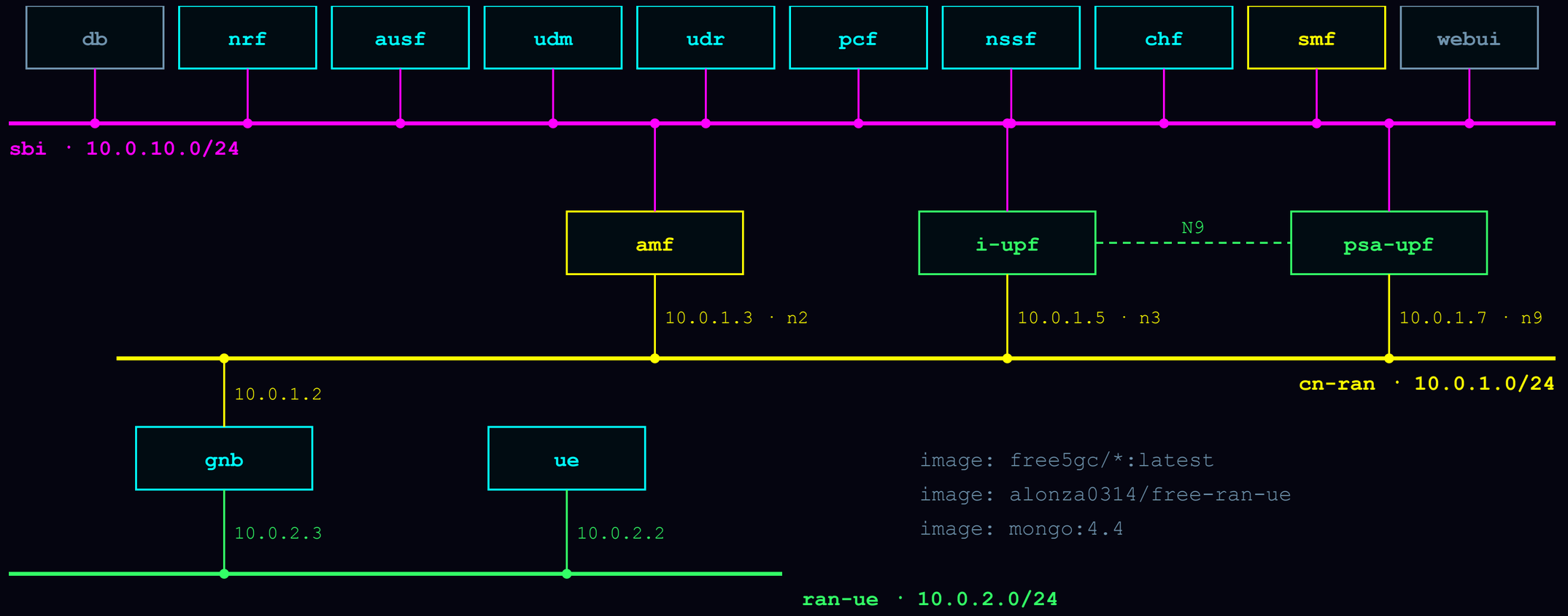


image: free5gc/*:latest
image: alonza0314/free-ran-ue
image: mongo:4.4

/> COMPOSE 檔案結構

```
services:          # 每個 container
  db:    ...
  nrf:   ...
  amf:   ...
  ...
  gnb:   ...
  ue:    ...

networks:          # 自訂網路
  sbi:     ...
  cn-ran:  ...
  ran-ue:  ...

volumes:           # 具名 volume
  dbdata:
```

services
16 個 container：mongo、11 個 free5GC NF、webui、gnb、ue

networks
3 條 bridge 網路，對應 SBI、N2/N3/N9、Uu 三個平面

volumes
dbdata 存 MongoDB 資料，down 之後訂閱資料還在

service name = hostname
同網路內可直接用名字互連，例如 mongodb://db

/> SERVICE 範例：AMF

```
amf:
  container_name: amf
  image: free5gc/amf:latest
  command: ./amf -c ./config/amfcfg.yaml
  volumes:
    - ./config/amfcfg.yaml:/free5gc/config/amfcfg.yaml
    - ./cert:/free5gc/cert
  environment:
    GIN_MODE: release
  networks:
    sbi:
      aliases: [amf.free5gc.org]
    cn-ran:
      ipv4_address: 10.0.1.3
      aliases: [n2.free5gc.org]
  depends_on:
    - nrf
```

image / command
用哪個 image、啟動時執行什麼

volumes
把 Host 上的設定檔與憑證掛進去，改設定不用重 build

networks.aliases
額外的 DNS 名稱，設定檔裡寫 amf.free5gc.org

ipv4_address
N2 要給 gNB 連，所以釘固定 IP

depends_on
NRF 先起來，AMF 才能去註冊

/> SERVICE 範例：I-UPF / PSA-UPF

```
i-upf:
  image: free5gc/upf:latest
  command: bash -c "./upf-iptables.sh &&
           ./upf -c ./config/upfcfg.yaml"
  volumes:
    - ./config/ULCL/i-upfcfg.yaml:/free5gc/config/upfcfg.yaml
    - ./script/upf-iptables.sh:/free5gc/upf-iptables.sh
  cap_add:
    - NET_ADMIN
  networks:
    sbi:
      aliases: [i-upf.free5gc.org]
    cn-ran:
      ipv4_address: 10.0.1.5
      aliases: [n3.free5gc.org]

psa-upf:  # 同 image，換 psa-upfcfg.yaml
          # cn-ran: 10.0.1.7 / n9.free5gc.org
```

同一個 image

I-UPF、PSA-UPF 只差在掛進去的設定檔

cap_add: NET_ADMIN

要設 iptables NAT、建立 GTP 網路介面

upf-iptables.sh

啟動前先設定封包轉送與 NAT 規則

Host 前提

Host 必須先裝好 gtp5g kernel module

/> SERVICE 範例：SMF（ULCL）

```
smf:
  image: free5gc/smf:latest
  command: ./smf -c ./config/smfcfg.yaml -u ./config/uerouting.yaml
  volumes:
    - ./config/ULCL/smfcfg.yaml:/free5gc/config/smfcfg.yaml         # 兩顆 UPF 的拓樸
    - ./config/ULCL/uerouting.yaml:/free5gc/config/uerouting.yaml   # 分流規則
  depends_on: [nrf, i-upf, psa-upf]
```



W6 講的 ULCL，在 compose 裡就是「多一個 UPF container + 換一份 SMF 設定」。

/> SERVICE 範例：GNB / UE

```yaml
gnb:
  image: alonza0314/free-ran-ue:latest
  command: ./free-ran-ue gnb -c ./gnbcfg.yaml
  networks:
    cn-ran:
      ipv4_address: 10.0.1.2
    ran-ue:
      ipv4_address: 10.0.2.3
  depends_on: [amf, smf, i-upf, ...]
```

```yaml
ue:
  image: alonza0314/free-ran-ue:latest
  command: ["tail", "-f", "/dev/null"]
  cap_add:
    - NET_ADMIN
  devices:
    - /dev/net/tun:/dev/net/tun
  networks:
    ran-ue:
      ipv4_address: 10.0.2.2
```

gNB 同時接兩條網路：往核心網走 cn-ran，往 UE 走 ran-ue

UE 只用 `tail -f /dev/null` 保持存活，進去後手動啟動 UE

UE 要建立 ueTun0，需要 TUN 裝置與 NET_ADMIN

/> NETWORKS & VOLUMES

```
networks:
  sbi:
    ipam:
      driver: default
      config:
        - subnet: 10.0.10.0/24
    driver_opts:
      com.docker.network.bridge.name: docker-sbi
  cn-ran:    # 10.0.1.0/24 → docker-cn-ran
  ran-ue:    # 10.0.2.0/24 → docker-ran-ue

volumes:
  dbdata:
```

ipam.subnet

固定網段，才能幫 AMF、UPF、gNB 釘 IP

bridge.name

Host 上看得到 docker-cn-ran 這張網卡

抓封包

`sudo tcpdump -i docker-cn-ran` 就能看 NGAP、GTP-U

dbdata

MongoDB 資料存在 volume，down -v 才會清掉

/> COMPOSE 操作

`docker compose -f <file> up -d`	建立網路、volume，背景啟動全部
`docker compose ps`	列出這個 compose 的 container
`docker compose logs -f amf`	追蹤某個 service 的 log
`docker compose exec ue bash`	進入某個 service 的 container
`docker compose restart smf`	改完設定檔後重啟單一 service
`docker compose pull`	把 image 更新到最新的 latest
`docker compose down`	停止並刪除 container 與網路
`docker compose down -v`	連 volume（資料庫）一起刪掉

不是預設檔名 `docker-compose.yaml` 時，每個指令都要加 `-f <file>`。

/> DEMO：ULCL 跑起來

```
$ git clone https://github.com/free-ran-ue/fru-compose && cd fru-compose
$ docker compose -f docker-compose-ulcl.yaml up -d
$ docker compose -f docker-compose-ulcl.yaml ps      # 確認 16 個都 Up

# 瀏覽器開 http://localhost:5000 → Webconsole 新增 UE 訂閱資料

$ docker exec -it ue bash
(ue)$ ./free-ran-ue ue -c ./uecfg.yaml               # 註冊 + 建 PDU Session

# 另一個 terminal
$ docker exec -it ue ping -I ueTun0 1.1.1.1 -c 5

$ docker compose -f docker-compose-ulcl.yaml down
```

使用說明：free-ran-ue.github.io/doc-user-guide

/> COMPOSE 的好處

01
宣告式
寫「要什麼」，不用寫「怎麼做」

02
一鍵起停
up / down 管理整套環境

03
內建 DNS
用 service 名稱與 aliases 互連

04
網路隔離
依介面切開 SBI、N2/N3、Uu

05
版本控管
YAML 進 git，改動可追蹤

06
可重現
同一份檔案，到誰電腦都一樣

/> COMPOSE 的極限

只能跑在一台機器上

機器掛了，container 不會自己搬到別台

沒有滾動更新，更新 image 要先停再起

擴充副本、負載平衡都要自己處理

多台機器、多個團隊共用時很難管理

→ **需要能管理「一群機器」的 Kubernetes**

$ PART 2 · ORCHESTRATION AT SCALE

/> KUBERNETES

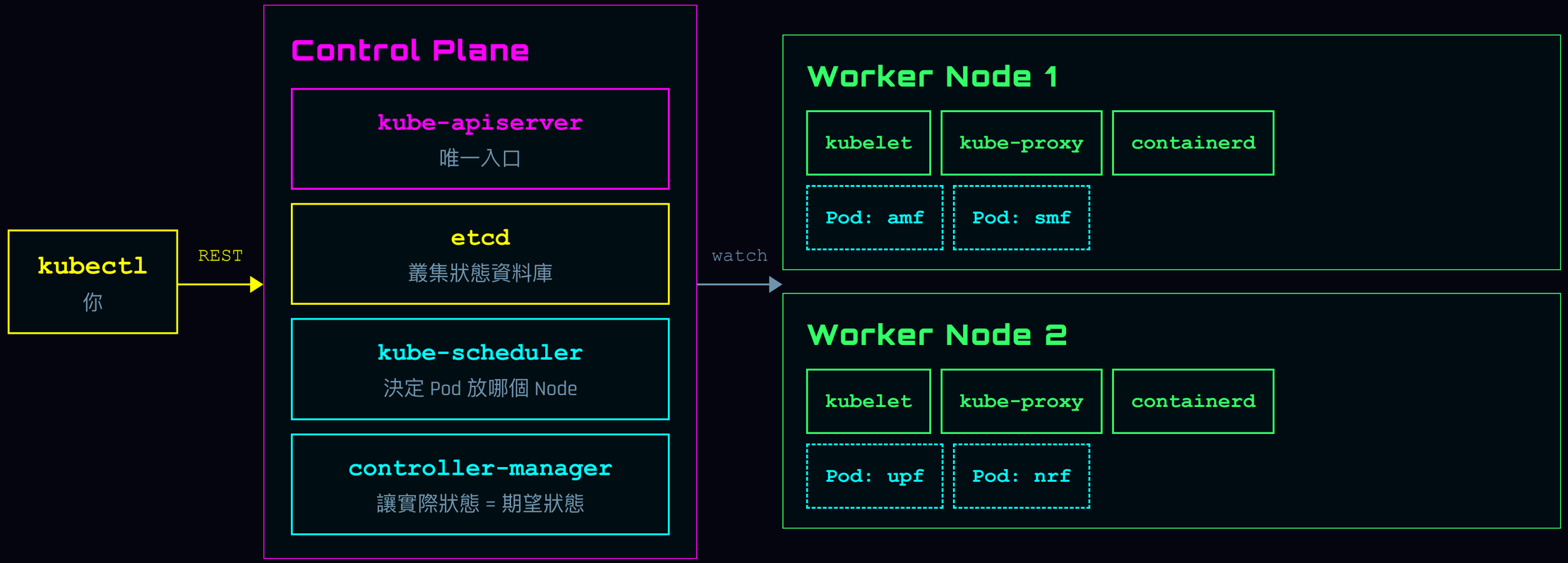
希臘文「舵手」-- 幫你管理一整群機器上的容器

/> 題外話：為什麼叫 K8S？

Kubernetes	中間 8 個字母	K8s
internationalization	中間 18 個字母	i18n
localization	中間 10 個字母	l10n
accessibility	中間 11 個字母	a11y
observability	中間 11 個字母	o11y

這種「頭 + 中間字母數 + 尾」的縮寫叫 numeronym。i18n = 國際化，l10n = 在地化。

/> KUBERNETES 架構



/> 宣告式：期望狀態 VS 實際狀態



你不下「啟動 3 個」的指令，而是宣告「我要 3 個」—— 這就是 Pod 掛掉會自動補回來的原因（self-healing）。

/> NODE

一台實體機或 VM 就是一個 Node

Control Plane Node：跑管理元件

Worker Node：真正跑 Pod 的地方

每台都有 kubelet、kube-proxy、container runtime

microk8s 單機：兩種角色在同一台

```
$ kubectl get nodes -o wide
NAME       STATUS   ROLES    VERSION
free5gc    Ready    <none>   v1.28.x
```

kubelet
Node 上的代理人：收到 Pod 規格，叫 runtime 把 container 跑起來

kube-proxy
負責把 Service 流量轉到正確的 Pod

/> POD

K8s 的最小部署單位

一個 Pod 可以有多個 container

同 Pod 共用 IP、port 空間與 volume

Pod 是短暫的：重建後 IP 會變

通常不直接建 Pod，交給 Deployment

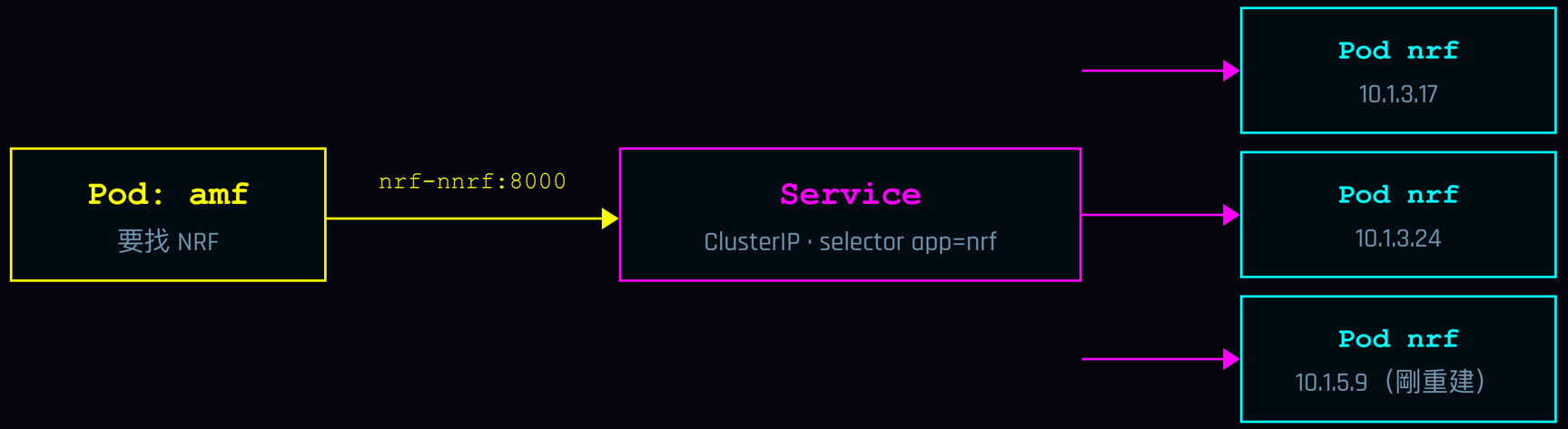
```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nrf
  labels:
    app: nrf          # 給 Service 找
spec:
  containers:
    - name: nrf
      image: free5gc/nrf:latest
      command: ["./nrf", "-c",
                "./config/nrfcfg.yaml"]
      ports:
        - containerPort: 8000
```

/> DEPLOYMENT → REPLICASET → POD




```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nrf
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nrf
  template:           # Pod 樣板
    metadata:
      labels:
        app: nrf
    spec:
      containers: ...
```

/> SERVICE：固定的入口



Pod IP 一直變，Service 提供不變的 IP 與 DNS 名稱

用 label selector 找到後面的 Pod，順便做負載平衡

完整 DNS：`nrf-nnrf.free5gc.svc.cluster.local`

/> SERVICE 類型

TYPE	誰連得到	FREE5GC-HELM 例子
ClusterIP	只有叢集內部（預設）	nrf-nnrf:8000（SBI）
NodePort	任一 Node IP + 30000-32767	amf-n2 SCTP 38412 → 31412
LoadBalancer	外部 LB 給一個對外 IP	雲端環境 / loxilb chart
Headless	不給 ClusterIP，直接拿 Pod IP	資料庫等有狀態服務

N2 是 SCTP、N3 是 GTP-U —— 電信介面不只是 HTTP，這也是 5GC 上 K8s 比一般 Web 服務麻煩的地方。

/> ETCD

分散式 key-value 資料庫

存所有物件：Pod、Service、ConfigMap...

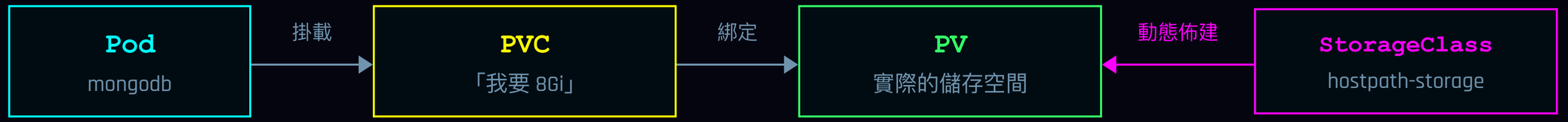
只有 kube-apiserver 會直接讀寫它

用 Raft 共識，多數決才算寫入成功

備份 etcd ＝ 備份整個叢集



/> STORAGE



Pod 刪掉，PV 裡的資料還在 → 訂閱資料不會消失

NF 的 amfcfg.yaml 這類設定，用 ConfigMap 掛進 Pod

free5gc-helm 的 MongoDB 用 hostPath，並綁在指定 Node

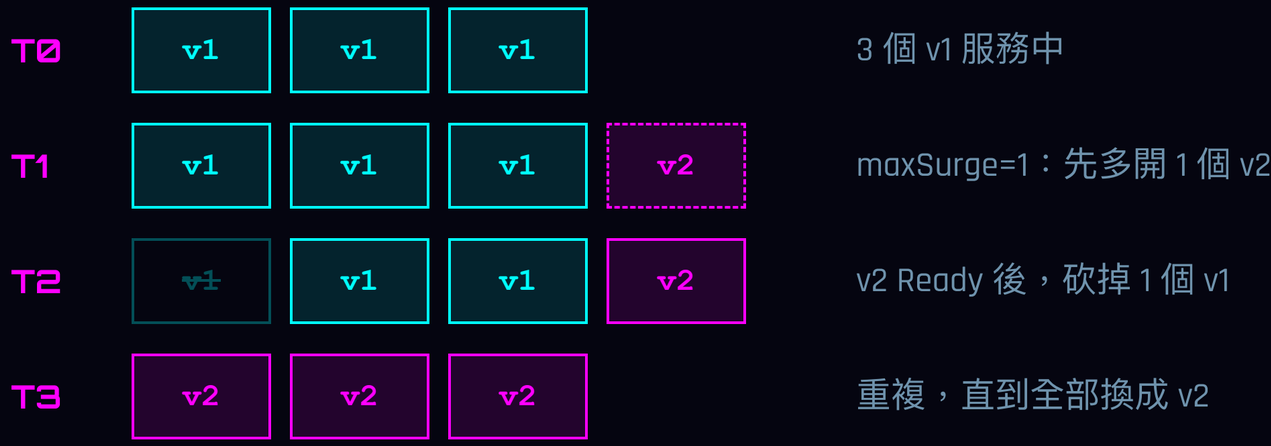
所以 Ansible 會改 values.yaml 的 path 與 node name

/> KUBECTL 基礎操作

`kubectl get pods -n free5gc -o wide`	列出 Pod、所在 Node 與 IP
`kubectl get svc,deploy -n free5gc`	一次看多種資源
`kubectl describe pod <pod>`	看事件：為什麼起不來
`kubectl logs -f <pod>`	追蹤 NF 的 log
`kubectl exec -it <pod> -- sh`	進到 container 裡
`kubectl apply -f nrf.yaml`	宣告 / 更新期望狀態
`kubectl delete pod <pod>`	刪掉後 Deployment 會補回
`kubectl port-forward svc/<svc> 5000:5000`	把服務接到本機

`-n` 指定 namespace；free5gc-helm 全部裝在 `free5gc` 這個 namespace。

/> 滾動更新 ROLLING UPDATE



maxUnavailable=0 → 整個過程隨時都有 3 個可用 Pod，服務不中斷；出問題可以 rollout undo 回上一版。

/> 滾動更新操作

```
$ kubectl set image deploy/nrf nrf=free5gc/nrf:v4.0.1 -n free5gc   # 換 image 觸發更新
$ kubectl rollout status  deploy/nrf -n free5gc                    # 看進度
$ kubectl rollout history deploy/nrf -n free5gc                    # 版本紀錄
$ kubectl rollout undo    deploy/nrf -n free5gc                    # 回到上一版
$ kubectl rollout restart deploy/nrf -n free5gc                    # 同 image 重啟
```

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

deploy 名稱以 get deploy 為準

AMF / SMF 有 UE 狀態，換版可能斷線

無狀態的 NF 最適合滾動更新

$ PART 2 · PACKAGE MANAGER

/> HELM

Kubernetes 的 apt / npm —— 把幾十個 YAML 打包成一個可重用的 Chart

/> 為什麼需要 HELM？

一個 NF ≈ Deployment + Service + ConfigMap + …，乘上 15 個 NF → 幾十個 YAML，換一個 IP 要改好幾處。

PACKAGE

Chart

一包 K8s 樣板 + 預設值，可以包含子 chart

CONFIG

values.yaml

可調參數集中在一處，用 -f / --set 覆寫

INSTANCE

Release

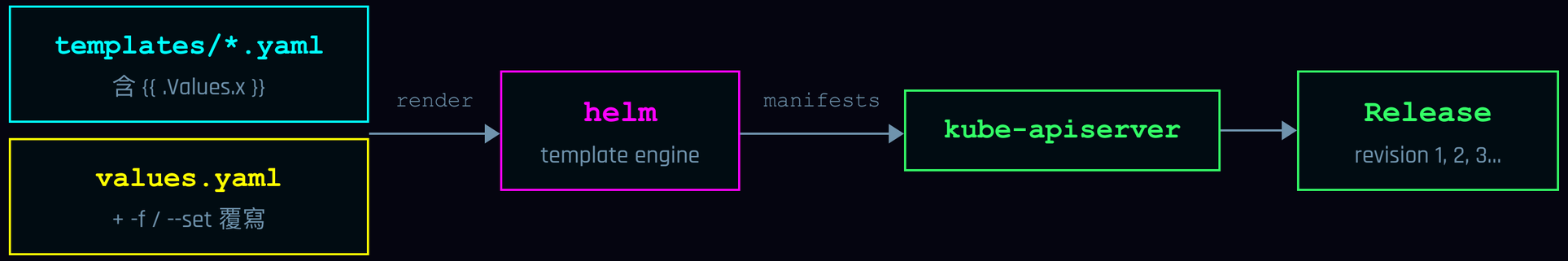
chart 裝進叢集的一個實例，有 revision 可回滾

STORE

Repository

存放 chart 的地方，就像 image 的 registry

/> HELM 怎麼運作




```
# templates/deployment.yaml
image: "{{ .Values.image.name }}:{{ .Values.image.tag }}"
# values.yaml
image: { name: free5gc/amf, tag: v4.0.1 }
# 渲染結果
image: "free5gc/amf:v4.0.1"
```

/> FREE5GC-HELM CHART 結構

```
free5gc-helm/charts/
├── free5gc/            # umbrella chart
│   ├── Chart.yaml      # 子 chart 清單
│   ├── values.yaml     # global 設定
│   └── charts/
│       ├── free5gc-amf/
│       ├── free5gc-smf/
│       ├── free5gc-upf/
│       ├── ... nrf ausf udm udr
│       │       pcf nssf chf nef ...
│       └── mongodb-15.6.0/
├── free-ran-ue/        # submodule
├── ueransim/
├── packetrusher/
└── loxilb/
```

```yaml
# free5gc/Chart.yaml
apiVersion: v2
name: free5gc
version: 1.1.7
appVersion: latest
dependencies:
- name: mongodb
  condition: deployMongoDb
- name: free5gc-amf
  condition: deployAmf
- name: free5gc-smf
  condition: deploySmf
...
```

每個 NF 一個子 chart；`condition` 讓你用 values 開關要不要部署某個 NF。

/> VALUES.YAML（GLOBAL 節錄）

```yaml
global:
  name: free5gc
  userPlaneArchitecture: ulcl  # single | ulcl
  uesubnet: 10.60.0.0/16
  nrf:
    service:
      name: nrf-nnrf
      type: ClusterIP
      port: "8000"
  amf:
    multus:
      enabled: false
    service:
      ngap:
        name: amf-n2
        port: 38412
        protocol: SCTP
```

userPlaneArchitecture
一個值切換單 UPF 或 ULCL 拓樸

uesubnet
UE IP pool，等同 compose 裡 SMF 的設定

nrf.service
所有 NF 用這個 Service 找 NRF

multus.enabled
開啟後 N2/N4… 走獨立網卡，而非 Pod 預設網路

/> HELM 操作

```
$ cd free5gc-helm/charts
$ helm install free5gc-helm ./free5gc -n free5gc            # 安裝，Release 叫 free5gc-helm
$ helm upgrade --install free5gc-helm ./free5gc -n free5gc  # 沒有就裝、有就升級
$ helm upgrade free5gc-helm ./free5gc -n free5gc \
    --set global.userPlaneArchitecture=single               # 覆寫參數
$ helm list -n free5gc                                      # 列出 Release
$ helm get values free5gc-helm -n free5gc                   # 看目前用的 values
$ helm history  free5gc-helm -n free5gc                     # revision 紀錄
$ helm rollback free5gc-helm 1 -n free5gc                   # 回到 revision 1
$ helm template ./free5gc | less                            # 只渲染，不安裝
$ helm uninstall free5gc-helm -n free5gc                    # 移除整套
```

$ PART 2 · RELIABILITY

/> HIGH AVAILABILITY

任何一個元件掛掉，服務都不能停

K8S 提供的 HA 手段

REPLICAS

多副本 + 分散

replicas > 1，用 anti-affinity 分到不同 Node

PROBES

健康檢查

liveness 失敗重啟；readiness 失敗先摘掉流量

CONTROL PLANE

3 台以上 master

apiserver 多台 + etcd quorum，管理層不單點

PDB

PodDisruptionBudget

維護、升級 Node 時，至少保留 N 個 Pod

5GC 做 HA 的挑戰

元件	有什麼狀態	難點
NRF / NSSF…	幾乎無狀態	多副本 + Service 即可
AMF	UE context、NAS 安全	N2 是 SCTP 長連線
SMF	PDU Session、PFCP	換 Pod 要能接回 session
UPF	GTP tunnel、轉送規則	依賴 Node 的 gtp5g、網卡
MongoDB	訂閱資料	需要 replica set

狀態要搬出 Pod 才能真正 HA —— 這就是 3GPP 定義 AMF Set、UDSF 的原因。

$ PART 3 · HANDS-ON

/> FREE5GC-HELM

用 Ansible 一鍵把一台空機器，變成跑著 free5GC 的 K8s

/> 環境需求

CPU

AMD 或 Intel（x86_64）

OS

Ubuntu 20.04 / 22.04 / 24.04 / 25.04

Tools

git

建議

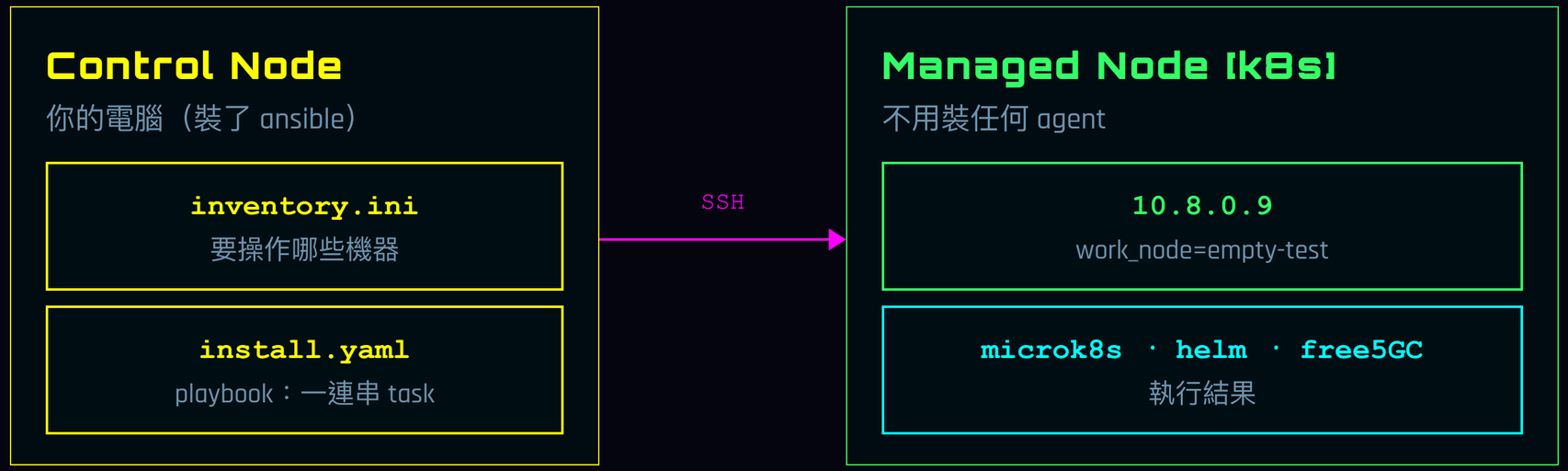
空白的機器；已裝過的請參考 free5gc-helm 文件

指南：free5gc.org/guide/quick-setup-helm

安裝 Ansible

```
$ sudo apt update
$ sudo apt install -y software-properties-common
$ sudo add-apt-repository --yes --update \
      ppa:ansible/ansible
$ sudo apt install -y ansible
```

/> ANSIBLE 是什麼？



Agentless：只要能 SSH，就能自動化

冪等：同一份 playbook 重跑，結果一樣

/> 設定目標機器並執行

```
$ git clone https://github.com/free5gc/free5gc
$ cd free5gc
$ vim ansible-helm/inventory.ini

[k8s]
<host IP> work_node=<host name>

$ ./quick-setup-helm.sh
Enter SSH user: <user>
SSH password:
BECOME password:
```

quick-setup-helm.sh 做的事

```
ansible-playbook \
  -i ansible-helm/inventory.ini \
  ansible-helm/install.yaml \
  -u "$USER" \
  --ask-pass --ask-become-pass
```

work_node

填 node 的 hostname，MongoDB 會綁在這台

--ask-become-pass

安裝要 sudo，會問 sudo 密碼

/> INSTALL.YAML 做了什麼（1/2）

01

timedatectl set-ntp true

校正系統時間，避免憑證與 log 時間錯亂

02

snap install microk8s --channel=1.28

輕量、單機就能跑的 K8s

03

snap install kubectl + microk8s config

安裝 kubectl，產生 ~/.kube/config

04

snap install helm

Helm 套件管理工具

05

install k9s

終端機裡的 K8s 管理介面

06

microk8s enable hostpath-storage community multus

儲存、社群 addon、多網卡 CNI

/> INSTALL.YAML 做了什麼（2/2）

07 `--allowed-unsafe-sysctls "net.ipv4.ip_forward"`
允許 Pod 開啟 IP forwarding（UPF 需要）

08 `build & install gtp5g v0.9.14`
UPF 使用的 kernel module，裝在 Node 上

09 `git clone free5gc-helm`
下載 chart

10 `replace mongodb path / node name`
改 values.yaml 的 hostPath 與 work_node

11 `kubectl create ns free5gc`
建立 namespace

12 `helm upgrade --install free5gc-helm / ueransim`
部署核心網與 RAN 模擬器

/> 為什麼要這些前置作業？

gtp5g

kernel module

- UPF 用它處理 GTP-U
- container 共用 kernel → 裝在 Node

multus

CNI plugin

- 讓 Pod 有第二張以上的網卡
- N2 / N3 / N4 / N6 可以分開

hostpath-storage

StorageClass

- 提供 MongoDB 用的 PV
- 資料存在 Node 的目錄

ip_forward sysctl

kubelet 參數

- 預設不允許 Pod 改這個值
- UPF 要轉送封包，必須打開

/> 啟動與檢查

```
$ helm list -n free5gc
NAME            NAMESPACE    STATUS
free5gc-helm    free5gc      deployed
ueransim        free5gc      deployed

$ kubectl get pods -n free5gc
# amf / smf / upf / nrf / ... / mongodb
# 全部 STATUS = Running

$ k9s -A          # 所有 namespace
```

Ansible 結果
最後的 PLAY RECAP 要 failed=0

Pending
多半是 PV 沒綁上 → 檢查 work_node

CrashLoopBackOff
看 logs；UPF 常見原因是 gtp5g

Init 卡住
NF 在等 NRF 或 MongoDB 先 Ready

/> K9S：終端機裡的 K8S 介面

`:pod`	切到 Pod 列表	`:svc`	切到 Service 列表
`:deploy`	切到 Deployment	`0`	顯示所有 namespace
`/`	過濾，例如 /amf	`l`	看選取 Pod 的 log
`s`	進入 container shell	`d`	describe，看事件
`ctrl-d`	刪除（會被重建）	`?`	所有快捷鍵說明

k9s 是對 kubectl 的包裝：每個按鍵背後都是一個 kubectl 指令，用來觀察與除錯非常直覺。`:q` 離開。

/> RAN / UE 測試：FREE-RAN-UE

```
$ cd free5gc-helm
$ git submodule update --init                    # 取得 charts/free-ran-ue
$ helm upgrade --install fru ./charts/free-ran-ue -n free5gc
$ kubectl get pods -n free5gc | grep fru

$ kubectl exec -it -n free5gc fru-freeranue-ue-xxxxxxxxxx-xxxxx -- sh
/free-ran-ue # ./free-ran-ue ue -c config/ue.yaml

# 另開一個 terminal，exec 進同一個 Pod
/free-ran-ue # ping -I ueTun0 1.1.1.1 -c 5
PING 1.1.1.1 (1.1.1.1) from 10.60.0.1 ueTun0: 56(84) bytes of data.
64 bytes from 1.1.1.1: icmp_seq=1 ttl=48 time=3.99 ms
```

UE 拿到 10.60.0.1 —— 正好落在 values.yaml 的 uesubnet 10.60.0.0/16。

/> COMPOSE VS HELM

	DOCKER COMPOSE	K8S + HELM
規模	單一台機器	多 Node 叢集
設定	一份 compose YAML	Chart + values.yaml
自癒	restart policy（本機）	controller 自動補 Pod
更新	停掉再起	滾動更新 + rollback
網路	bridge network + IP	Service、CNI、Multus
適合	開發、課堂實驗、CI	多機部署、接近正式環境

/> 總結

Dockerfile → build → push → run，image 讓環境可重現

CI 每天 release latest，打 tag 時 release 固定版本

Compose 用一份 YAML 在單機跑起整套 5GC + RAN

K8s：Node、Pod、Service、etcd，宣告式又能自癒

Helm 把一堆 YAML 包成 chart，用 values 客製

free5gc-helm：Ansible 一鍵安裝，k9s 觀察